

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “ Парадигмы и конструкции языков программирования”

Отчет по ЛР № 3
“Биквадратные уравнения”



Выполнил:
Студент группы ИУ5-36Б
Калмыков А.Ю.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Листинг кода

```
import math

class QuarticEquationSolver:
    def __init__(self):
        self.a = None
        self.b = None
        self.c = None
        self.roots = []

    def _get_coef(self, prompt, allow_zero):
        while True:
            try:
                coef_str = input(prompt)
                coef = float(coef_str)
                if not allow_zero and coef == 0.0:
                    print("Ошибка: коэффициент не  
может быть равен нулю. Попробуйте снова.")
                    continue
                return coef
            except ValueError:
                print("Ошибка: введите действительное  
число. Попробуйте снова.")

    def input_coefficients(self):
        self.a = self._get_coef('Введите коэффициент  
A (не равный нулю): ', allow_zero=False)
        self.b = self._get_coef('Введите коэффициент  
B: ', allow_zero=True)
        self.c = self._get_coef('Введите коэффициент  
C: ', allow_zero=True)

    def _calculate_roots(self):
```

```

result = []
D = self.b**2 - 4 * self.a * self.c

if D == 0.0:
    y = -self.b / (2.0 * self.a)
    if y > 0.0:
        result.extend([math.sqrt(y), -
math.sqrt(y)])
    elif y == 0.0:
        result.append(0.0)
elif D > 0.0:
    sqD = math.sqrt(D)
    y1 = (-self.b + sqD) / (2.0 * self.a)
    y2 = (-self.b - sqD) / (2.0 * self.a)

    if y1 > 0.0:
        result.extend([math.sqrt(y1), -
math.sqrt(y1)])
    elif y1 == 0.0:
        result.append(0.0)

    if y2 > 0.0:
        result.extend([math.sqrt(y2), -
math.sqrt(y2)])
    elif y2 == 0.0 and y1 != 0.0:
        result.append(0.0)

self.roots = sorted(set([round(r, 5) for r in
result]))

def solve(self):
    self._calculate_roots()

```

```
def output_results(self):
    count = len(self.roots)
    if count == 0:
        print('Нет корней')
    else:
        roots_str = ', '.join(str(r) for r in
self.roots[:-1])
        if count == 1:
            print(f'Один корень:
{self.roots[0]}')
        elif count == 2:
            print(f'Два корня: {self.roots[0]} и
{self.roots[1]}')
        else:
            roots_str = ', '.join(str(r) for r in
self.roots[:-1])
            print(f'{count} корней: {roots_str} и
{self.roots[-1]}')

def main():
    solver = QuarticEquationSolver()
    solver.input_coefficients()
    solver.solve()
    solver.output_results()

if __name__ == "__main__":
    main()
```

Скриншоты работы программы

```
PS C:\Users\vrema\Documents\Github\IU5_3sem_PiKYAP\Lab_3> & C:/Users/vrema/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vrema/Documents/Github/IU5_3sem_PiKYAP/Lab_3/lab_3.py
Введите коэффициент A (не равный нулю): 1
Введите коэффициент B: -5
Введите коэффициент C: -36
Два корня: -3.0 и 3.0
PS C:\Users\vrema\Documents\Github\IU5_3sem_PiKYAP\Lab_3> & C:/Users/vrema/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vrema/Documents/Github/IU5_3sem_PiKYAP/Lab_3/lab_3.py
Введите коэффициент A (не равный нулю): 0
Ошибка: коэффициент не может быть равен нулю. Попробуйте снова.
Введите коэффициент A (не равный нулю): 1
Введите коэффициент B: -5
Введите коэффициент C: bukovki
Ошибка: введите действительное число. Попробуйте снова.
Введите коэффициент C: -37
Два корня: -3.01272 и 3.01272
PS C:\Users\vrema\Documents\Github\IU5_3sem_PiKYAP\Lab_3> █
```